```
    P ←
randomPermutation(P)
       #6
  return P
```

#1 Method `randomStep` takes a candidate solution `P` (a point in the problem space, i.e. a permutation of the list of vertices in the graph), and computes a transition to a new neighboring state.
#2 Choose a random value between 0 and 1 (excluded).
#3 Based on the number chosen and some static probabilities, decides which transition to apply.
#4 Swap two adjacent vertices.
#5 Swap two arbitrary vertices. Notice that the two indices might in theory be equal: rather than overcomplicating the code, we can risk a collision, which just means no swap is performed. Especially for large lists, chances are low enough anyway.
#6 Replace current state with a random new state (a random permutation).

These results show a couple more things that are worth highlighting:

- Even on a small case (10 vertices) and running thousands of iterations, we get arbitrarily close to the best solution, but we don’t always get the best result: that’s a calculated risk with heuristics.
- Finding the best configuration for an optimization algorithm requires time, experience… and sometimes a bit of luck.